# Response Letter

**ICSE 2027 – Paper #1111**

Structure-Aware Cross-Dialect SQL Transfer for DBMS Fuzzing Seed Construction

We thank the reviewers for their valuable comments and for recognizing the practical significance of MORPH. Below, we respond first to the four requested changes in the meta-review and then to the individual reviewers. Our revision focuses on accurately positioning the technical contribution, explaining the relationship to prior work, and making the evaluation and artifact reproducible.

**Working draft – October 22, 2026.** Reviewer concerns below are condensed for readability. Responses describe the intended revision and evidence already reported in the supplied rebuttal; they do not certify that manuscript edits or experimental verification have been completed. Blue notes track outstanding evidence and locations and must be resolved before submission.

## Replies to Requested Changes

**Requested Change 1:** Correct the description of SEDAR's comment-before-mutation and restore-after-mutation mechanism, and explain the actual methodological differences.

**Reply:** We agree that describing SEDAR's mutability refinement as permanent removal or "destructive sanitization" mischaracterizes its design. SEDAR temporarily hides fragments that the downstream fuzzer cannot parse and restores those fragments after mutation. This mechanism is intended to retain target-specific constructs while enabling mutation of the remaining input.

MORPH's intended distinction is that it repairs incompatible constructs before the transferred seeds are passed to the downstream fuzzer. Repaired constructs can then participate in mutation within the fuzzer's supported representation. Neither strategy guarantees semantic equivalence after mutation. Any example in which restoration causes an execution error must be presented as an observed behavior of the evaluated implementation under a specific mutated context, rather than as a general consequence of SEDAR's algorithm.

We will align the motivation, figure captions, method description, and related work with this explanation. The motivating example will identify the transferred input, hidden fragment, mutation, restored statement, and observed error, subject to verification of the original trace.

**Planned manuscript location:** Introduction; Background and Motivation; Feedback-Driven Adaptive Fuzzing; Related Work. Add final page/line references after integration.

**Pending before submission:** Verify the motivating trace and the restoration behavior of the actual SEDAR executable; replace every inaccurate characterization in the clean manuscript.

**Requested Change 2:** Moderate the novelty claims and distinguish MORPH from SEDAR, RISE, QTRAN, and CrackSQL, explaining what AQT contributes to DBMS fuzzing.

**Reply:** We agree that structural abstraction, query decomposition, dialect retrieval, and local translation are established techniques. We do not claim that MORPH introduces these techniques individually, or that it is the first system to transfer complete test cases or adapt seeds to downstream fuzzers.

The contribution we aim to establish is coordinated transfer of an ordered, stateful test sequence. AQT combines statement templates, explicit forward read-after-write dependencies, and cumula-

tive schema/type context to guide target-side instantiation and repair across statements. For example, a target-side schema change must be reflected in the context used to instantiate subsequent references to the changed object. The method description must make the use of this information explicit, rather than present AQT only as an annotated AST.

Our discussion of prior work will acknowledge SEDAR's complete-test workflow and source-schema extraction; RISE's treatment of self-contained queries and procedural artifacts; CrackSQL's functionality-based processing, syntax embedding, and local-to-global translation; and QTRAN's dialect-aware translation for metamorphic testing. The comparison will focus on the transfer unit, state/context handling, feedback, and intended testing task.

The compositional baselines and seed-count controls reported in the rebuttal evaluate the complete transfer pipelines. They do not isolate AQT or the explicit graph as the sole cause of improvement. We will make this limitation explicit and distinguish it from the evidence supplied by the context-propagation ablation.

**Planned manuscript location:** Abstract; Introduction and contributions; AQT definition and algorithms; Table I; Related Work; Conclusion. Add final page/line references after integration.

**Pending before submission:** Check each prior-work description against its primary source and each claimed use of the graph against implementation traces; integrate the verified comparisons.

**Requested Change 3:** Clarify how MORPH differs from and complements SmartFuzz, and acknowledge the limited scope of the additional comparison.

**Reply:** As described in our rebuttal, SmartFuzz constructs DBMS-specific seeds using DBMS documentation and historical bug-triggering inputs, whereas MORPH transfers existing test cases from other DBMSs. These approaches exploit different input resources and may provide complementary seed sources. Their combination has not been evaluated in the reported comparison, so we do not claim an experimentally demonstrated benefit from combining them.

The rebuttal reports a DuckDB experiment using 300 seeds from each method, the same GRIFFIN configuration, and a 24-hour budget. SmartFuzz and MORPH covered 81,090 and 100,903 branches, respectively. These measurements apply to that target, sample, and configuration. They do not establish superiority across DBMSs, seed budgets, or the entire SmartFuzz framework. The revision will include these conditions and explicitly discuss the limitations of the comparison.

**Planned manuscript location:** Evaluation; Discussion and threats to validity; Related Work. Add final page/line references after integration.

**Pending before submission:** Verify corpus versions, sampled files, random seeds, run counts, and coverage logs; confirm SmartFuzz's input resources and workflow against its paper.

**Requested Change 4:** Clarify the provenance of the SEDAR binary, including conflicting reports about public availability, and provide sufficient configuration information for reproducibility.

**Reply:** We recognize that the phrase "original released binary" in our rebuttal is insufficient to establish the origin or public availability of the executable used in the evaluation. Availability to the authors and public availability are different facts and must be reported separately.

The revision requires a precise provenance record: the actual acquisition channel and date, the release/version identifier if available, the binary checksum, runtime dependencies, configuration and invocation, and any changes to the binary or surrounding adapter. It must also explain how the comment-and-restore behavior was checked. The artifact should provide the materials that can be shared and state any remaining access restrictions accurately.

The current materials do not establish these facts sufficiently to complete this response. We will finalize the provenance statement from the original acquisition and execution records, and assess whether any affected comparison requires correction if the implementation differs from the one described in the paper.
**Planned manuscript location:** Experimental Setup; Data Availability; artifact README and baseline configuration records. Add final page/line references after integration.
**Pending before submission:** Supply the actual acquisition record, version/hash, adapter changes, commands, and restoration evidence. Do not replace this note with an unsupported claim of public release.

# Replies to Reviewer A

**Comment 1.1:** Characterize novelty and the relationship to SEDAR, CrackSQL, RISE, and QTRAN accurately.
**Reply:** Thank you for identifying these distinctions. We agree with the methodological precedents and the corrections outlined in Requested Change 2. In particular, SEDAR already handles complete test cases, and RISE is not restricted to a single SQL statement or to SELECT queries. CrackSQL should not be described as mining mutation patterns with genetic algorithms. We will revise these descriptions and focus the contribution on cross-statement coordination under evolving target-side context.
**Planned manuscript location:** Introduction; Table I; Related Work. Add final page/line references after integration.

**Comment 1.2:** Explain the independent role of the dependency graph beyond the original statement order and context propagation.
**Reply:** We agree that all edges in the stated graph follow the original order, so the original sequence is already a valid topological ordering. A stable topological sort alone therefore does not establish an additional capability or benefit.

Our rebuttal describes the graph as selecting dependency-relevant statements and object-state summaries during instantiation and repair. This explanation needs to be reflected in the implementation, algorithm, and example consistently. If the implementation uses the graph only for ordering, the claim must be limited accordingly. The existing ablation removes context propagation and cannot isolate the graph's independent contribution.

We will specify the operations that consume graph edges and separate evidence for cumulative context from evidence for graph-guided selection. An independent graph benefit will be claimed only if supported by an appropriately controlled comparison.
**Planned manuscript location:** AQT definition; extraction/instantiation algorithms; component ablation; Discussion. Add final page/line references after integration.
**Pending before submission:** Inspect the graph consumers and context-construction traces; decide whether a separate graph ablation is needed for the retained claim.

**Comment 1.3:** Define object-state summaries and narrow the scope of dependency preservation.
**Reply:** The summaries described in our rebuttal contain object existence, table definitions, column names and types, constraints, and DDL-induced schema changes. They are maintained incrementally as statements update the relevant metadata. We will distinguish source-side constraints from the evolving target-side context and specify the update procedure.

The formal dependency relation captures forward read-after-write relationships, and the context records schema/type information. These mechanisms do not establish preservation of all data, transaction, or other state dependencies. We will narrow the corresponding claims throughout the paper.
**Planned manuscript location:** Formal Definition of AQT; context propagation; limitations. Add final page/line references after integration.
**Pending before submission:** Match the listed summary fields and update semantics to the implemented representation.

**Comment 1.4:** Rename "Semantic Correctness" and evaluate source-to-target result or state equivalence.
**Reply:** We agree that execution without a runtime error measures execution validity, not semantic equivalence. We will rename the metric to *Execution Validity* and define its unit, denominator, and treatment of unsuccessful executions explicitly. Claims about equivalent syntax, preserved intent, or high-fidelity transfer will be limited to the evidence available.

In the rebuttal, we reported the following comparison on 100 sampled MySQL-to-DuckDB test cases:

| Method | Result equivalence | State equivalence |
|---|---|---|
| SQLGlot | 14% | 56% |
| RISE | 34% | 65% |
| SEDAR | 38% | 69% |
| MORPH | 52% | 74% |

These sampled results are supplementary evidence and do not demonstrate general semantic equivalence or equivalence after mutation. The revision will specify the sampling procedure and comparison protocol, including how failed transfers, unordered results, and target-specific differences are handled.
**Planned manuscript location:** Evaluation metrics; supplementary equivalence evaluation; Discussion; Abstract and Conclusion. Add final page/line references after integration.
**Pending before submission:** Verify the 100-case sample, denominators, state-equivalence definition, and result-comparison records before treating the table as final.

**Comment 1.5:** Explain how translation baselines were adapted to the target DBMSs and applied to complete test sequences.
**Reply:** The rebuttal describes CrackSQL and RISE as translating statements individually and reassembling outputs in their original order without additional schema context, while SEDAR receives complete tests and extracts source-schema context. These choices and their implications for fairness need explicit documentation.

For every reported source–target pair, we will identify the baseline implementation, supported dialects, target documentation or knowledge base, model, prompts, rules, manual adaptations, and

context supplied. Unsupported configurations must remain distinguishable from measured failures. The provenance and restoration behavior of SEDAR will be documented as described in Requested Change 4.
**Planned manuscript location:** Experimental Setup; baseline capability/configuration table; artifact. Add final page/line references after integration.
**Pending before submission:** Resolve differences between the local response drafts concerning CrackSQL's supported targets; verify adapters and configuration records.

**Comment 1.6:** Report repeated-run statistics, bug aggregation, campaign budgets, and offline cost.
**Reply:** Our rebuttal states that coverage is averaged over five independent runs, while bug counts are unions across runs after stack-trace deduplication and manual verification. It also distinguishes the seven-day-per-target campaign in RQ1, which reports 28 bugs, from the five repeated 24-hour runs in RQ2, which report 22 bugs. We will make these different budgets and aggregation procedures explicit.

The revision will report dispersion and confidence intervals per relevant method/target, together with the statistical test, comparison, and sampling unit for any significance claim. Cost reporting will distinguish USD per source test suite from USD per retained seed, and cumulative processing time from elapsed wall-clock time. A single aggregate variance or p-value without a specified comparison is insufficient.
**Planned manuscript location:** Experimental Setup; RQ1/RQ2; coverage and ablation tables; cost table. Add final page/line references after integration.
**Pending before submission:** Recompute statistics from per-run records; verify campaign instance counts and resource budgets; reconcile per-suite/per-seed units across response drafts.

**Comment 1.7:** Extend the context-propagation ablation to additional source–target pairs.
**Reply:** We agree that the PostgreSQL-to-DuckDB ablation alone provides limited evidence of generality. An extension should keep the transfer corpus, model, target configuration, and evaluation budget consistent while comparing the complete pipeline with the context-disabled variant on additional representative pairs. Execution validity and, where measured, downstream coverage should be reported separately. No completed extension is established by the materials currently available for this response.
**Planned manuscript location:** Component ablation; threats to external validity. Add final page/line references after integration.
**Pending before submission:** Confirm the final rebuttal commitment, select the required additional pairs, and add verified results or clearly state the remaining scope limitation.

**Comment 1.8:** Document source–target allocation and validation of automatically generated translation rules.
**Reply:** We will report how source test files are allocated to each target and distinguish unique source files from repeated transfers across source–target pairs. The rule-validation procedure must identify the checks performed automatically, the conditions triggering manual review, and the corrections made. The reported 136 corrected rules should be traceable to these records; parser acceptance alone must not be described as proof of semantic equivalence.
**Planned manuscript location:** Experimental Setup; rule generation and validation; manual-effort table; artifact. Add final page/line references after integration.
**Pending before submission:** Verify allocation lists and the rule-correction log; reconcile parser-validation and execution-validation descriptions.

# Replies to Reviewer B

**Comment 2.1:** Explain what AQT adds beyond existing AST/IR representations and include compositional baselines.

**Reply:** The intended contribution is the use of dependency and cumulative schema/type information to coordinate transfer across an ordered test sequence, rather than the existence of an AST-like representation itself. Requested Change 2 and Comment 1.2 describe the required clarification and its evidential limits.

The rebuttal reports the following DuckDB comparison with a shared downstream fuzzer, GRIFFIN, and a 24-hour budget:

| Translator + GRIFFIN | Branch coverage | Unique bugs |
|---|---|---|
| SQLGlot | 108,385 | 0 |
| RISE | 124,147 | 0 |
| SEDAR | 114,876 | 1 |
| MORPH | 144,822 | 3 |

This comparison evaluates the complete translation pipelines. It complements the seed-count controls below but does not isolate the independent contribution of AQT's graph or any individual translation component.

**Planned manuscript location:** AQT explanation; baseline comparison; Discussion. Add final page/line references after integration.

**Pending before submission:** Verify input lists, model/context settings, valid seed counts, and run aggregation for this table.

**Comment 2.2:** Discuss SmartFuzz and compare its seed construction with MORPH where feasible.

**Reply:** Requested Change 3 provides the task distinction and the limited DuckDB comparison reported in the rebuttal. We will integrate that discussion into Related Work and report the comparison with its exact sample and budget. The difference in input resources is relevant to interpreting the results and to possible complementarity.

**Planned manuscript location:** Related Work; Evaluation; threats to validity. Add final page/line references after integration.

**Comment 2.3:** Correct the characterization of SEDAR and compare mutation handling under the same downstream fuzzer.

**Reply:** We agree with the correction in Requested Change 1. Our rebuttal additionally reports a comparison using the same 300 translated seeds and the respective mutation-handling strategies with GRIFFIN for 24 hours, with coverage improvements of 27.2% and 15.5% on DuckDB and MonetDB. Before using this result to attribute an effect specifically to mutation handling, we need to verify that the seed files and all other settings were identical.

The sampled equivalence results in Comment 1.4 concern transferred tests. They cannot estab-

lish equivalence of mutated queries or show that one mutation-handling strategy guarantees it.
**Planned manuscript location:** Mutation-handling explanation; controlled comparison; limitations. Add final page/line references after integration.
**Pending before submission:** Verify the shared 300-seed manifest and the handling-specific experiment; do not substitute the separate seed-count comparison for this experiment.

**Comment 2.4:** Define the capability criteria in Table I and reconsider the classifications of SQLGlot, jOOQ, and CrackSQL.
**Reply:** We agree that multi-statement parsing/transpilation, schema-state interpretation, and coordinated test-case transfer are different capabilities. The table should define each column and provide primary-source or experimental evidence for each entry. SQLGlot's support for multi-statement scripts and jOOQ's schema metadata tracking must be acknowledged, while claims about CrackSQL's DML support must be checked.

We will distinguish explicit support, partial or indirect support, and unavailable evidence. In particular, lack of documentation must not be treated as evidence that a capability is absent. The table will use qualified entries where a simple binary classification would be misleading.
**Planned manuscript location:** Background and Motivation; Table I and its caption/notes. Add final page/line references after integration.
**Pending before submission:** Verify entries against the specific software versions and primary sources used in the comparison.

**Comment 2.5:** Control seed corpus size and determine whether gains arise from more seeds or from the transfer method.
**Reply:** We agree that a larger retained corpus can confound the comparison. The rebuttal reports equal-count experiments on DuckDB and MonetDB with 100, 200, and 300 seeds, a shared GRIFFIN setup, and a 24-hour budget:

| Target | Seeds | SEDAR branches | MORPH branches |
|---|---|---|---|
| DuckDB | 100 | 49,745 | 82,197 |
| DuckDB | 200 | 58,938 | 92,390 |
| DuckDB | 300 | 68,499 | 100,903 |
| MonetDB | 100 | 20,406 | 27,025 |
| MonetDB | 200 | 28,173 | 33,006 |
| MonetDB | 300 | 29,948 | 35,703 |

These results suggest that seed quantity alone does not explain the observed pipeline-level difference. Equal counts do not imply equal seed composition, and the experiment does not assign the full gain specifically to AQT. The revised evaluation will report the number of retained valid seeds for each method, the original corpus, the sampling procedure, and the repeated-run protocol.
**Planned manuscript location:** Experimental Setup; seed-count-controlled evaluation; threats to validity. Add final page/line references after integration.
**Pending before submission:** Check whether each number is a single run or an aggregate; verify sampling and validity criteria, and retain the distinction between equal counts and identical seeds.

**Comment 2.6:** Clarify translation overhead and compare MORPH's end-to-end time and cost with SEDAR.

**Reply:** We agree that "Cost/TS" and "Time/TS" require explicit units and denominators. Our rebuttal reports total offline costs of approximately USD 96 for MORPH and USD 144 for SEDAR across the evaluated transfers. However, the local response drafts differ in their per-suite and per-seed units. These values should be reconciled from the records before a final cost table is included.

The revision will report input size, retained seed count, LLM usage/cost, preprocessing and repair time, total cumulative time, and parallel execution settings where available. This will make the tradeoff between generation cost and processing time interpretable under a common protocol.

**Planned manuscript location:** Experimental Setup; translation-overhead table; Discussion. Add final page/line references after integration.

**Pending before submission:** Reconcile all cost and time entries from logs; distinguish source test suites, target-specific transfers, and retained valid seeds.

**Comment 2.7:** Provide accessible bug evidence and triggering test cases in the artifact.

**Reply:** The rebuttal states that triggering test cases were added to the anonymous artifact. We will check that the materials are accessible and identify the corresponding DBMS versions and reproduction procedures. Developer-confirmed status, fixes, and reported-but-unconfirmed bugs should remain distinguishable. Public report links and anonymous evidence should be managed consistently with the ongoing review requirements.

**Planned manuscript location:** Bug results; Data Availability; artifact README. Add final page/line references after integration.

**Pending before submission:** Check anonymous access and the completeness of the bug-evidence index; do not treat the presence of a link as proof of reproduction.

# Replies to Reviewer C

**Comment 3.1:** Tighten semantic-validity claims and provide a small differential result/state check.

**Reply:** We agree. Comment 1.4 describes the change to *Execution Validity* and the sampled equivalence results reported in the rebuttal. The revision will separate execution success from behavioral equivalence and use coverage and verified bug findings as the primary evidence of fuzzing utility. Repair by substitution or commenting may change behavior, and the claims must reflect that limitation.

**Planned manuscript location:** Metric definitions; transfer evaluation; Abstract; Discussion; Conclusion. Add final page/line references after integration.

**Comment 3.2:** Explain baseline configurations, unsupported targets, all-zero bug results, and the scope of the SQLancer comparison.

**Reply:** The revision will identify the baseline versions, commands, seed corpora, adapters, target support, and resource budgets. Unsupported targets must not be interpreted as zero performance. The reported 528% improvement over SQLancer is based only on the commonly supported DuckDB and ClickHouse targets and must be described with that scope.

The zero-bug rows record the outcomes of the evaluated campaigns. Fixes to previously known bugs may be relevant, but without supporting analysis they cannot be asserted as the established cause of the zero counts. The seed controls in Comment 2.5 provide additional context for com-

paring the evaluated pipelines.
**Planned manuscript location:** Experimental Setup; coverage/bug tables and captions; Discussion; artifact. Add final page/line references after integration.
**Pending before submission:** Verify support matrices and configurations, coverage instrumentation, bug deduplication, and the denominator used for aggregate improvements.

**Comment 3.3:** Resolve the inconsistency between 0.776 in the text and 0.748 in the transfer-results table.
**Reply:** Our rebuttal identifies 0.748 as the intended MORPH value for the comparison in question, with 0.314 for RISE. We will verify these against the corresponding data and make the text and table consistent. We will also check the remaining numerical summaries, including average versus maximum coverage improvements and percentage versus percentage-point differences.
**Planned manuscript location:** Transfer-results text/table; Abstract; Introduction; Conclusion. Add final page/line references after integration.
**Pending before submission:** Confirm the intended experiment and denominator before applying the numerical correction throughout.

**Comment 3.4:** Discuss SQLRight, sqllogictest, and seed-selection work, and clarify the SQLRight comparison.
**Reply:** We will explain the distinction between DBMS test-case transfer, logic-bug testing, cross-engine result validation, and fuzzing seed prioritization. These related tasks provide useful context but are not interchangeable evaluation goals.

The rebuttal reports a DuckDB comparison with SQLRight under a 10-hour budget. Its target adaptation, configuration, and coverage measurement need explicit documentation. The result must be labeled as a separate, limited comparison and must not be pooled with the 24-hour main evaluation. The differing numerical summaries in the local drafts will be reconciled before inclusion.
**Planned manuscript location:** Related Work; supplementary baseline comparison; limitations. Add final page/line references after integration.
**Pending before submission:** Verify SQLRight's evaluated implementation and adaptation, the 10-hour logs, and the primary references for all added work.

**Comment 3.5:** Explain novelty as a justified combination of established techniques rather than overstating the individual components.
**Reply:** We agree. Requested Change 2 describes the revised positioning: AQT is intended to coordinate structure, dependencies, and evolving schema/type context across an ordered test sequence during instantiation and repair. The revision should explain the capability enabled by that coordination and support it with appropriately scoped evidence, while acknowledging the established constituent techniques.
**Planned manuscript location:** Abstract; contributions; method overview; Related Work; Conclusion. Add final page/line references after integration.

**Comment 3.6:** Clarify how the 28-bug full campaign relates to the 22-bug 24-hour comparison.
**Reply:** As explained in Comment 1.6, the rebuttal assigns these totals to different budgets: seven days per target for RQ1 and five repeated 24-hour runs for RQ2. We will explicitly identify the aggregation and overlap of these bug sets, the number of instances, and the computational budget. The two totals should not be interpreted as results from the same campaign or added together.
**Planned manuscript location:** Experimental Setup; RQ1/RQ2; bug-table captions. Add final page/line references after integration.
**Pending before submission:** Verify campaign logs and the mapping of unique bugs across RQ1 and RQ2.